Leyendas SV — Plan de arquitectura (equipo de 2)

Nov 6, 2026 · @Que puedes hacer ?

1. Resumen

En dos días construimos un RPG de investigación jugable de inicio a fin: un pueblo salvadoreño, 4 NPC con IA local (Nemotron) y un misterio con 3 finales. Dos ingenieros trabajan en paralelo sobre un contrato de API fijo, así que el código se une sin conflictos.

- **El juego:** el jugador recorre el pueblo, habla libremente (texto o voz) con la Siguanaba, el Cipitío, el Cadejo y la señora de la pupusería, junta pistas y acusa al culpable.
- **Lo que lo hace distinto:** cada NPC recuerda lo que le dijiste, tiene un nivel de confianza contigo y comparte chismes con el resto del pueblo. Si le mientes a uno, otro puede saberlo.
- **Tracks que cubre:** AI Game Dev (el juego), Generative Assets (retratos y escenas), Narratives & Agents (los NPC).
- **Costo:** cero por uso. Todo corre en la PC del equipo, con la API gratuita de NVIDIA como respaldo.

Alcance cerrado (no se amplía hasta tener la versión jugable): 1 mapa, 4 NPC, 6 pistas, 3 finales, en español.

Extras solo si sobra tiempo, en este orden: voz, escenas animadas, multijugador cooperativo.

2. Stack de IA

El cerebro de los NPC es **Nemotron 3 Nano 4B** corriendo local en Ollama. Todo lo demás también es local y gratis, y la API gratuita de NVIDIA queda como respaldo con el mismo código.

Parte	Herramienta	Dónde corre	Tamaño	Por qué
Diálogo de NPC (principal)	Nemotron 3 Nano 4B, tag `nemotron-3-nano:4b`	GPU, Ollama	2.8 GB (Q4_K_M)	NVIDIA, rápido, cabe de sobra en 12 GB; la familia lista español entre sus idiomas

Parte	Herramienta	Dónde corre	Tamaño	Por qué
Diálogo (alternativa local)	Nemotron Nano 9B v2, `hf.co/bartowski/nvidia_NVIDIA-Nemotron-Nano-9B-v2-GGUF:Q4_K_M`	GPU, Ollama	6.5 GB	Más calidad si el 4B se queda corto en español
Respaldo en la nube	Nemotron 3 Super, `nvidia/nemotron-3-super-120b-a12b`	API de NVIDIA (build.nvidia.com)	0 local	Gratis con créditos de registro; compatible con OpenAI; sirve para la laptop sin GPU NVIDIA
Voz a texto	faster-whisper, modelo `small`, int8	CPU	0 VRAM	No le quita GPU al modelo
Texto a voz	Piper: `es_MX-claude-high` (voz femenina) y `es_MX-ald-medium` (masculina)	CPU	0 VRAM	Offline, licencia MIT
Retratos y escenas	SDXL en ComfyUI	GPU, por lotes	~8 GB	Se genera con el modelo de diálogo apagado
Memoria y estado	SQLite	Disco	1 archivo	Sin servidor que instalar
IA para programar	Claude, un chat por ingeniero (sección 10)	Nube	—	Solo para desarrollar; no corre dentro del juego

Descartados: Nemotron 3 Nano 30B pesa 24 GB y pide 30 GB de RAM, no cabe. Nemotron Mini 4B fue diseñado por NVIDIA para NPCs de videojuegos, pero está optimizado para inglés. Ese dato sí sirve en el pitch: la familia Nemotron nació pensando en NPCs.

Presupuesto de VRAM (12 GB) durante la demo: solo Nemotron en GPU (3 a 7 GB con contexto). Whisper y Piper en CPU. ComfyUI cerrado. Nunca correr ComfyUI y el juego a la vez.

Reglas del modelo de diálogo:

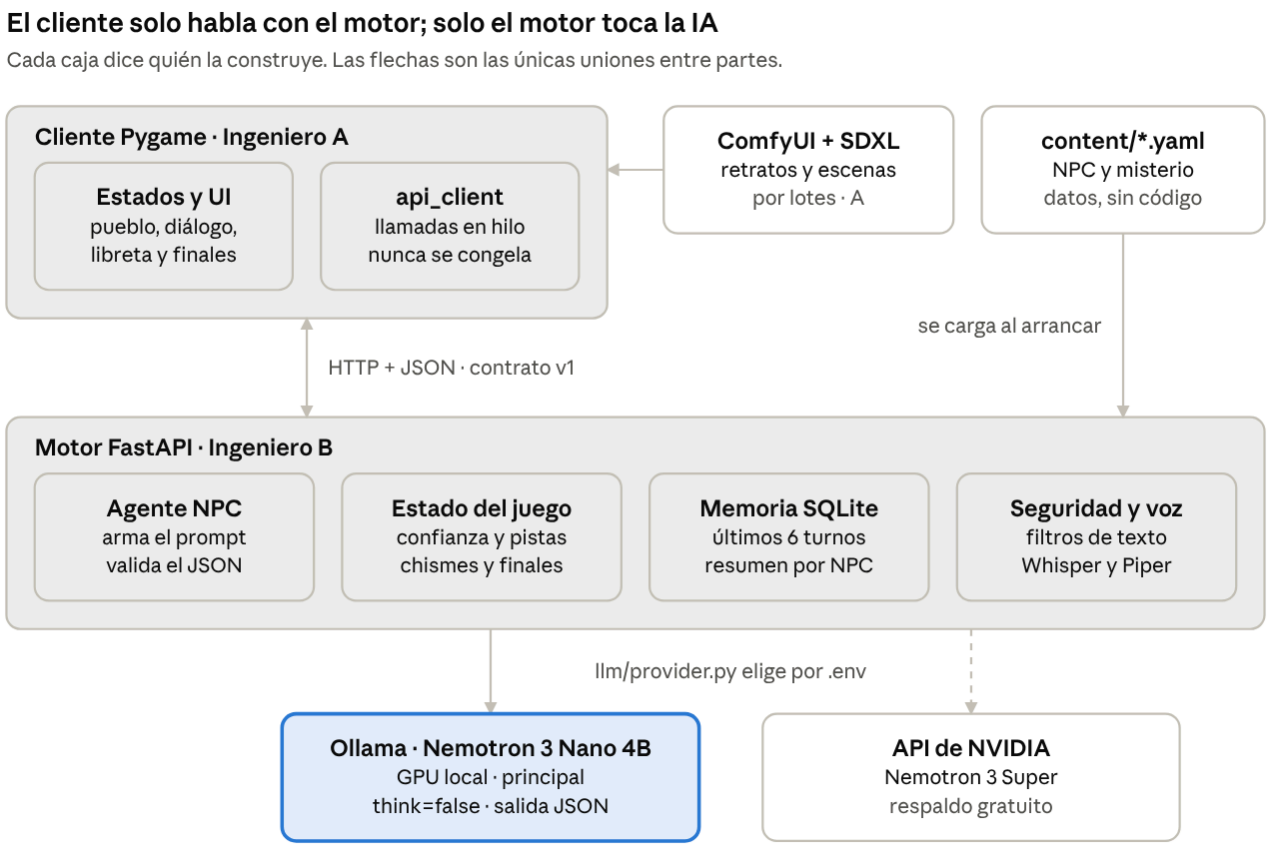
1. Razonamiento desactivado (`think: false` en Ollama). El modelo responde directo, sin pensar en voz alta, y baja la latencia.
2. `num_predict` 150, `temperature` 0.7, contexto 4096, salida en JSON con esquema (`format` de Ollama).

3. `keep_alive: -1` y una llamada de calentamiento al arrancar el servidor, para que la primera respuesta en la demo no tarde.
4. El modelo se elige en `.env` (`LLM_PROVIDER`, `LLM_MODEL`). Cambiar de 4B a 9B o a la nube no toca código.
5. **Prueba de arranque:** 10 preguntas a la Siguanaba. Se queda el modelo que responda en menos de 3 s con buen español.

Fuentes: Ollama, nemotron-3-nano:4b · GGUF de Nemotron Nano 9B v2 · Ollama, Nemotron 3 Nano 30B · Nemotron Mini 4B · API de NVIDIA · Voces Piper es_MX

3. Arquitectura

Dos procesos en la misma PC: el cliente Pygame (Ingeniero A) y el motor FastAPI (Ingeniero B). El cliente nunca llama al modelo directamente; todo pasa por el contrato de la sección 5.



arquitectura · cliente, motor e IA

Recorrido de una conversación:

1. El jugador escribe (o habla) y el cliente envía `POST /talk` desde un hilo. En pantalla aparece la animación de espera.

2. El motor carga la ficha del NPC, su memoria, la confianza actual, las pistas visibles para esa confianza y los chismes.
3. `prompt_builder.py` arma el prompt y `provider.py` llama a Nemotron con razonamiento apagado y esquema JSON.
4. `parser.py` valida la respuesta y `filters.py` la revisa. Si algo falla, sale una frase de respaldo.
5. El estado aplica `trust_delta`, agrega las pistas permitidas, crea chismes y guarda la memoria en SQLite.
6. El motor devuelve `TalkResponse` y el cliente muestra el texto letra a letra, el retrato con su emoción y la libreta actualizada.

4. Estructura del repositorio

Un solo repositorio (`fbclabs/leyendas-sv`) donde cada carpeta tiene un único dueño. Como nadie edita las carpetas del otro, unir el código no genera conflictos.

```
leyendas-sv/
├── README.md                cómo instalar y correr todo
├── .env.example             LLM_PROVIDER, LLM_MODEL, OLLAMA_URL,
NVIDIA_API_KEY
├── docs/
│   └── api.md               el contrato en palabras (copia de la sección 5)
├── shared/
│   └── contracts.py         modelos Pydantic de peticiones y respuestas
(API v1)
├── content/                 datos del juego, sin código
│   ├── mystery.yaml         verdad, culpable, pistas, finales
│   ├── npcs/                siguanaba.yaml, cipitio.yaml, cadejo.yaml,
nina_tere.yaml
│   └── art_prompts.yaml     prompts de retratos y escenas
├── server/                  motor de IA
│   ├── main.py              FastAPI y rutas
│   ├── llm/                 provider.py (Ollama | API NVIDIA), warmup.py
│   ├── agents/              npc_agent.py, prompt_builder.py, parser.py
│   ├── game/                state.py, trust.py, clues.py, gossip.py,
endings.py
│   ├── memory/              store.py (SQLite), summarizer.py
│   ├── safety/              filters.py
│   ├── voice/               stt.py (faster-whisper), tts.py (Piper)
│   ├── requirements.txt
│   └── tests/
├── client/                  juego Pygame
│   ├── main.py
```

```
|   ├── src/
|   |   ├── game.py           bucle principal y máquina de estados
|   |   ├── states/           menu, pueblo, dialogo, libreta, acusacion,
final
|   |   ├── entities/         player.py, npc.py
|   |   ├── systems/          api_client.py (hilo), audio.py, text_input.py
|   |   └── ui/               dialog_box.py, typewriter.py, portrait.py
|   ├── assets/               sprites, retratos, escenas, sonidos, fuentes
|   └── requirements.txt
├── tools/
|   ├── mock_server.py        servidor falso con el mismo contrato
|   ├── comfy_batch.py        genera los retratos desde art_prompts.yaml
|   └── smoke_llm.py          prueba de latencia y español del modelo
├── tests/
|   └── test_contract.py      valida mock y servidor real contra
shared/contracts.py
└── scripts/
    └── run_all.ps1           levanta Ollama, el servidor y el cliente
```

Carpeta o archivo	Dueño	Regla
`client/`, `content/art_prompts.yaml`, `tools/mock_server.py`, `tools/comfy_batch.py`	Ingeniero A	Solo A lo edita
`server/`, `content/npcs/`, `tools/smoke_llm.py`	Ingeniero B	Solo B lo edita
`shared/contracts.py`, `docs/api.md`, `content/mystery.yaml`, `tests/`	Los dos	Se cambia en pareja, en un PR que el otro aprueba
`README.md`, `scripts/`, `.env.example`	Los dos	Cambios pequeños, avisando por chat

Base técnica común: Windows 11, Python 3.12 (las librerías de voz e IA suelen ir un paso detrás de la última versión), un `.venv` en la raíz con los dos `requirements.txt`. El cliente sigue la misma estructura modular de Aventura Pixel (`main.py`, `src/game.py`, `states/`, `entities/`, `systems/`), así que se puede reutilizar código de ese juego.

Sugerencia de reparto: Fernando toma el rol A, porque ya domina Pygame. El compañero toma el rol B.

5. Contrato de integración

El cliente y el motor solo se hablan por HTTP con JSON en `http://127.0.0.1:8000`. Este contrato se escribe juntos en la primera hora y es lo único que los dos deben respetar para que el código se una sin sorpresas.

Ruta	Para qué	Envía	Recibe
`GET /health`	Saber si el motor y el modelo están listos	—	`{status, model, warm}`
`POST /session`	Empezar partida	`{player_name}`	`WorldState`
`GET /session/{session_id}`	Leer el estado del mundo	—	`WorldState`
`POST /talk`	Hablar con un NPC	`TalkRequest`	`TalkResponse`
`POST /accuse`	Acusar y cerrar la partida	`AccuseRequest`	`Ending`
`POST /stt` (extra)	Voz a texto	audio WAV (multipart)	`{text}`
`GET /tts/{audio_id}.wav` (extra)	Audio de la última respuesta	—	archivo WAV

```
# shared/contracts.py — API v1
from typing import Literal
from pydantic import BaseModel, Field

API_VERSION = "1"
Emotion = Literal["neutral", "feliz", "triste", "enojado", "asustado",
"sospechoso"]

class NpcState(BaseModel):
    npc_id: str
    trust: int = Field(ge=0, le=10)      # 0 desconfía, 10 confía
    met: bool = False
    last_emotion: Emotion = "neutral"

class WorldState(BaseModel):
    session_id: str
    api_version: str = API_VERSION
    clues_found: list[str] = []          # ids de content/mystery.yaml
    npcs: list[NpcState]
```

```
    rumors: list[str] = []                # chismes que circulan en el pueblo
    can_accuse: bool = False              # True con 3 o más pistas
    ended: bool = False

class TalkRequest(BaseModel):
    session_id: str
    player_id: str = "p1"                 # listo para multijugador
    npc_id: str
    text: str = Field(min_length=1, max_length=300)
    want_audio: bool = False

class TalkResponse(BaseModel):
    reply: str                            # máximo 3 frases
    emotion: Emotion
    trust: int
    trust_delta: int                      # de -2 a +2
    new_clues: list[str] = []
    audio_id: str | None = None
    source: Literal["llm", "cache", "fallback"] = "llm"
    state: WorldState

class AccuseRequest(BaseModel):
    session_id: str
    player_id: str = "p1"
    suspect_id: str
    evidence: list[str]                   # ids de pistas

class Ending(BaseModel):
    ending_id: Literal["verdad", "injusto", "abierto"]
    title: str
    text: str
    scene: str                            # archivo en client/assets/scenes/

class ApiError(BaseModel):
    error: str                            # p. ej. "model_not_ready",
"bad_npc"
    detail: str = ""
```

Reglas del contrato:

1. Solo cambios que suman: un campo nuevo siempre es opcional. Renombrar o borrar un campo rompe al otro y está prohibido durante el hackathon.
2. Todo cambio va en un PR que toca `shared/contracts.py` y `docs/api.md` a la vez, y lo aprueba el otro ingeniero.

3. `tests/test_contract.py` valida que el mock y el servidor real respondan con estos mismos modelos. Debe pasar antes de unir a `main`.
4. El cliente espera un máximo de 15 s. Si vence o recibe `ApiError`, muestra una frase de respaldo del NPC y el juego sigue.
5. `player_id` existe desde el día 1 aunque el juego sea de un jugador. Así, el multijugador cooperativo se puede agregar después sin romper nada.

6. Contenido: NPC, misterio y prompt

Todo el contenido vive en YAML, fuera del código. El servidor decide qué pistas puede revelar cada NPC según la confianza, y el modelo solo puede mencionar las que recibe en su prompt. Así un modelo pequeño no inventa pistas ni rompe el misterio.

Ficha de NPC (`content/npcs/siguanaba.yaml`, plantilla para los 4):

```
id: siguanaba
nombre: La Siguanaba
retrato: siguanaba            #
client/assets/portraits/siguanaba_<emocion>.png
voz: { modelo: es_MX-claude-high, velocidad: 0.95 }
personalidad: >
  Mujer hermosa de lejos y aterradora de cerca. Habla despacio, con frases
  cortas y poéticas. Seduce con palabras y castiga a los mentirosos.
como_habla: [tutea al jugador, le dice "mi niño" o "mi niña", nunca grita]
objetivo: Que el jugador culpe al Cipitío, aunque sea su propio hijo.
secreto: Vio a alguien con un canasto cerca del río la noche de la feria.
umbral_secreto: 7             # confianza necesaria para revelar el secreto
sabe:
  - id: pista_rio
    texto: Escuchó pasos que cruzaban el río a medianoche.
    confianza_min: 3
  - id: pista_canasto
    texto: El que cruzó llevaba un canasto pesado.
    confianza_min: 7
le_agrada: [halagos sinceros, hablar del río, flores]
le_molesta: [mentiras, que le pregunten por su hijo]
saludo: "¿Te perdiste, mi niño? El río canta bonito a estas horas..."
respaldo:
  - "El viento no me deja oírte..."
  - "Pregúntame otra cosa, mi niño."
```

Misterio (`content/mystery.yaml`): título, la verdad en un párrafo, culpable, las 6 pistas con su dueño, y qué hace falta para cada final. El ejemplo de arriba (feria del pueblo, algo desaparece, todos sospechan del Cipitío) es solo un punto de partida: lo deciden juntos

en el Bloque 0.

Final	Condición
`verdad`	Culpable correcto y al menos 3 pistas que lo prueban
`abierto`	Culpable correcto sin pruebas suficientes
`injusto`	Culpable equivocado

Plantilla de prompt (`server/agents/prompt_builder.py` la llena en cada turno):

```
Eres {nombre}, personaje de "Leyendas SV", un juego de misterio en un pueblo
de El Salvador.

QUIÉN ERES: {personalidad}
CÓMO HABLAS: {como_habla}
LO QUE QUIERES: {objetivo}
TU SECRETO: {secreto}  (no lo digas si tu confianza es menor que
{umbral_secreto})
LO QUE SABES Y PUEDES CONTAR: {pistas_visibles}
LO QUE RECUERDAS DEL JUGADOR: {resumen_memoria}
CHISMES DEL PUEBLO: {rumores}
TU CONFIANZA EN EL JUGADOR: {trust}/10
TE AGRADA: {le_agrada} · TE MOLESTA: {le_molesta}

REGLAS:
1. Responde en español sencillo de El Salvador, máximo 3 frases.
2. Nunca digas que eres una IA ni salgas del personaje.
3. Solo menciona pistas de "LO QUE SABES Y PUEDES CONTAR".
4. Si el jugador miente u ofende, baja la confianza. Si es amable o te ayuda,
súbela.
5. Nada de violencia gráfica ni contenido sexual. Si el jugador insiste,
cambia de tema como lo haría tu personaje.

Responde SOLO con este JSON:
{"reply": "...", "emotion":
"neutral|feliz|triste|enojado|asustado|sospechoso", "trust_delta": -2 a 2,
"mentions_clues": ["ids"]}
```

Cómo se procesa cada respuesta:

1. `parser.py` valida el JSON con Pydantic. Si falla, reintenta una vez y luego usa una frase de `respaldo` (`source: "fallback"`).

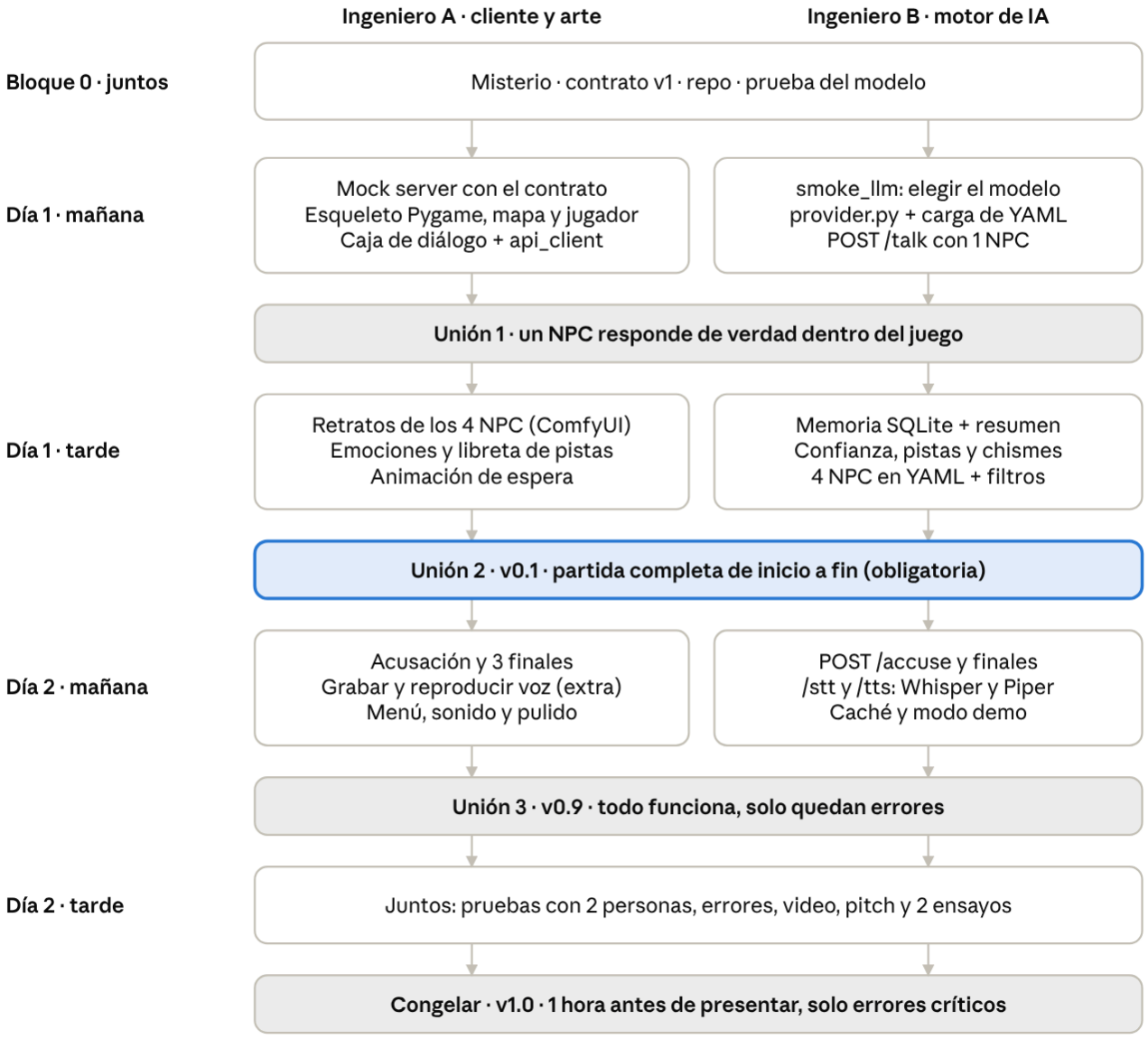
2. `mentions_clues` se cruza con las pistas visibles. Solo las que coinciden pasan a `new_clues`.
3. `trust_delta` se limita a −2..+2 y la confianza a 0..10.
4. Cada pista nueva crea un chisme para los otros 3 NPC (`gossip.py`). Ese es el momento "wow" de la demo: el pueblo se entera de lo que haces.
5. La memoria guarda los últimos 6 turnos completos y, cada 6 turnos, un resumen de 2 frases por NPC.

7. Reparto en paralelo y cronograma

Los dos avanzan en carriles separados y se juntan cuatro veces. En cada unión, `main` tiene que correr completo antes de seguir.

Dos carriles en paralelo que se unen cuatro veces

Cada unión exige que main corra de punta a punta antes de seguir



cronograma · 2 carriles, 4 uniones

Qué se prueba en cada unión:

- **Unión 1:** el cliente real contra el motor real; un NPC contesta en personaje en menos de 3 s.
- **Unión 2 (v0.1):** se juega de principio a fin: hablar con los 4 NPC, juntar 3 pistas, acusar y ver un final. Si no se logra, esa noche se recorta el alcance.
- **Unión 3 (v0.9):** todo el alcance funciona, incluido el modo demo. Desde aquí solo se corrigen errores.
- **Congelar (v1.0):** etiqueta final; la demo corre desde esa versión.

Ritmo de trabajo: 10 minutos de sincronía al inicio de cada bloque (qué terminó cada uno, qué bloquea). Si uno termina antes, ayuda al otro probando su parte, no editando su carpeta.

8. Flujo de Git y cómo se une el código

`main` siempre corre. Cada uno trabaja en ramas cortas dentro de sus carpetas y une a `main` en cada punto de unión, nunca al final del hackathon.

1. **Ramas:** `a/<tema>` para el Ingeniero A y `b/<tema>` para el B (por ejemplo, `a/dialog-box`, `b/talk-endpoint`). Una rama dura un bloque como máximo.
2. **Antes de empezar cada bloque:** `git pull --rebase origin main`.
3. **Commits pequeños** con prefijo: `feat(client): caja de diálogo`, `fix(server): JSON inválido`, `content: ficha del Cadejo`.
4. **PR a `main`:** el otro lo revisa en 5 minutos (que corra, que no toque carpetas ajenas). Se une con *squash*.
5. **Antes de unir:** `scripts/run_all.ps1` levanta todo y `pytest tests/test_contract.py` pasa.
6. **Etiquetas:** `v0.1` al cierre del día 1, `v0.9` a mediodía del día 2, `v1.0` la versión de la demo. Si algo se rompe, se vuelve a la última etiqueta.
7. **`.gitignore`:** `.env`, `.venv/`, `*.db`, salidas crudas de ComfyUI, modelos descargados. Los PNG finales de `client/assets/` sí se suben, porque la demo los necesita.
8. **Nunca** se sube la clave de NVIDIA ni otra credencial. Va solo en `.env`; en el repo queda `.env.example` con valores vacíos.

Desarrollo en paralelo sin esperar al otro: A trabaja contra `tools/mock_server.py` desde el primer minuto. B prueba el motor con `curl` o con la documentación automática de FastAPI en `/docs`. Cuando el motor real está listo, A cambia una sola variable (`API_URL`) y el cliente apunta al motor real.

9. Demo, riesgos y plan B

La demo corre en la PC con la RTX y no depende del Wi-Fi del evento. Cada riesgo conocido tiene una salida preparada antes de subir al escenario.

Riesgo	Cómo se nota	Plan B
Respuestas lentas	Más de 3 s por turno	Modelo 4B, máximo 150 tokens, calentamiento al arrancar; el cliente muestra animación de espera y texto letra a letra
Español pobre o raro	Falla la prueba de arranque	Cambiar en `.env` a Nemotron Nano 9B v2 o a la API de NVIDIA
El modelo devuelve JSON roto	`parser.py` falla	Salida con esquema (`format`), un reintento, luego frase de respaldo
El NPC se sale del personaje o dice algo inapropiado	Lo detecta `filters.py`	Se reemplaza por una frase de respaldo del NPC
GPU llena	El juego se congela con ComfyUI abierto	Cerrar ComfyUI; el arte se genera solo entre bloques
La PC falla en la demo	No arranca o se cuelga	Laptop con `LLM_PROVIDER=nvidia` (API gratuita), o apuntando a la PC por la red local; último recurso, el video grabado
La integración llega tarde	Algo no encaja en una unión	Mock server desde el minuto 1 y uniones cada medio día
El alcance se come el tiempo	Sin partida completa al cierre del día 1	Recortar a 3 NPC y 4 pistas esa misma noche
Reglas del hackathon	Duda de los organizadores	Confirmar en el Bloque 0 que se permiten modelos locales y generar todo el arte durante el evento

Modo demo (`DEMO_MODE=1` en `.env`): arranca una partida guardada con 2 pistas ya encontradas y la caché de respuestas activa. Así, en 3 minutos se ve la memoria, un chisme y la acusación, aunque algo falle.

Guion del pitch (3 min):

1. **Gancho (30 s):** "La Siguanaba te está mintiendo. ¿Por qué?"
2. **Demo en vivo (75 s):** un juez le habla a un NPC, el NPC recuerda lo dicho y otro NPC repite el chisme.

3. **Cómo funciona (45 s):** el diagrama de la sección 3; Nemotron corriendo en la PC, sin internet ni costo por uso.
4. **Cierre (30 s):** tres tracks cubiertos, identidad salvadoreña y el multijugador ya previsto en el contrato.

Lo que más suma con el jurado: que un juez escriba lo que quiera y el NPC responda en personaje. Ensayen que alguien intente romper al NPC, porque es lo primero que hará un juez.

10. Prompts para abrir un chat nuevo

Cada ingeniero abre su propio chat con Claude, pega el **prompt maestro** seguido del **prompt de su rol**, y adjunta este documento o `shared/contracts.py`. Los dos chats comparten el mismo contexto, así que el código que produzcan encaja.

Prompt maestro (lo pegan los dos):

```
Eres mi copiloto técnico en un hackathon de 2 días (The Game Studio, de
CUBO_ai). Somos 2 ingenieros trabajando en paralelo. Respóndeme en español,
paso a paso, explicando el porqué antes de cada acción, con código completo
listo para copiar.

PROYECTO: "Leyendas SV: el pueblo vivo". RPG de investigación en un pueblo
salvadoreño. El jugador habla libremente con 4 NPC: la Siguanaba, el Cipitío,
el Cadejo y la Niña Tere de la pupusería. Cada NPC es un agente de IA con
personalidad, objetivo, secreto, memoria y un nivel de confianza de 0 a 10.
Las pistas se desbloquean con la confianza y los NPC comparten chismes entre
ellos. Con 3 o más pistas el jugador acusa al culpable; hay 3 finales
(verdad, abierto, injusto).

RESTRICCIONES: IA gratuita y local. Windows 11, Python 3.12, PC con RTX serie
50 de 12 GB de VRAM y 32 GB de RAM. Alcance cerrado: 1 mapa, 4 NPC, 6 pistas,
3 finales.

STACK:
- Cliente: Pygame, estructura main.py, src/game.py, states/, entities/,
systems/, ui/.
- Motor: FastAPI en http://127.0.0.1:8000.
- Modelo de diálogo: Ollama con nemotron-3-nano:4b, think=false, salida JSON
con esquema (format). Alternativas elegidas por .env: Nemotron Nano 9B v2
local, o la API gratuita de NVIDIA (https://integrate.api.nvidia.com/v1,
compatible con OpenAI, modelo nvidia/nemotron-3-super-120b-a12b).
- Memoria y estado: SQLite. Voz (extra): faster-whisper small en CPU y Piper
es_MX en CPU. Arte: SDXL en ComfyUI, generado por lotes.
```

```
REPOSITORIO (monorepo fbclabs/leyendas-sv): shared/contracts.py (contrato
Pydantic, API v1), content/ (YAML de NPC y misterio), server/ (Ingeniero B),
client/ (Ingeniero A), tools/mock_server.py, tests/test_contract.py,
scripts/run_all.ps1.

CONTRATO (no lo cambies; si hace falta, propónme un campo nuevo opcional):
GET /health · POST /session · GET /session/{id} · POST /talk (TalkRequest ->
TalkResponse) · POST /accuse (AccuseRequest -> Ending) · POST /stt · GET
/tts/{audio_id}.wav
[PEGAR AQUÍ EL CONTENIDO DE shared/contracts.py]

REGLA DE DISEÑO CLAVE: el servidor decide qué pistas puede revelar cada NPC
según la confianza; el modelo solo puede mencionar las pistas que recibe en
su prompt.

REGLAS DE TRABAJO:
1. Solo toca las carpetas de mi rol. Nunca modifiques shared/ sin avisarme.
2. Código modular y pequeño, con type hints. Cada pieza debe poder correrse y
probarse sola.
3. Al terminar cada pieza, dame el comando para probarla y el mensaje de
commit.
4. Si algo pone en riesgo la latencia (menos de 3 s por respuesta) o el
alcance, avísame antes de seguir.
5. Primero que funcione de punta a punta; después, que se vea bonito.
```

Prompt del Ingeniero A (cliente y arte):

```
MI ROL: Ingeniero A, cliente Pygame y arte.
MIS CARPETAS: client/, content/art_prompts.yaml, tools/mock_server.py,
tools/comfy_batch.py.

ENTREGABLES EN ORDEN:
1. tools/mock_server.py en FastAPI, que cumpla el contrato con respuestas
fijas por NPC.
2. Esqueleto Pygame con estados: menu -> pueblo -> dialogo -> libreta ->
acusacion -> final.
3. Mapa del pueblo (tilemap simple), jugador que se mueve y NPC con zona de
interacción (tecla E).
4. Caja de diálogo con entrada de texto en español (acentos y ñ con el evento
TEXTINPUT), texto letra a letra y animación de espera.
5. systems/api_client.py: las llamadas HTTP van en un hilo con cola, para que
el juego nunca se congele; timeout de 15 s con frase de respaldo.
6. Retratos por NPC con 6 emociones, libreta de pistas, pantalla de acusación
y 3 finales.
7. tools/comfy_batch.py: genera los retratos en lote desde
content/art_prompts.yaml usando la API local de ComfyUI, con un estilo visual
único para todo el juego.
8. Extra: grabar voz con push-to-talk (sounddevice), enviarla a /stt y
reproducir el WAV de /tts.

Empieza por el punto 1 y espera mi confirmación antes de pasar al siguiente.
```

Prompt del Ingeniero B (motor de IA):

```
MI ROL: Ingeniero B, motor de IA (servidor).
MIS CARPETAS: server/, content/npcs/, tools/smoke_llm.py.

ENTREGABLES EN ORDEN:
1. tools/smoke_llm.py: mide latencia y calidad del español de nemotron-3-
nano:4b contra Nemotron Nano 9B v2 con 10 preguntas a la Siguanaba, con
think=false.
2. server/llm/provider.py: una interfaz única chat(messages, schema) con dos
implementaciones, Ollama nativo (think=false, format JSON, keep_alive=-1) y
OpenAI-compatible para la API de NVIDIA, elegidas por .env. Llamada de
calentamiento al arrancar.
3. Carga de content/npcs/*.yaml y agents/prompt_builder.py con la plantilla
de prompt del plan.
4. POST /talk: arma el prompt, llama al modelo, valida el JSON con Pydantic
(1 reintento, luego frase de respaldo), aplica confianza y pistas visibles.
5. memory/store.py en SQLite: últimos 6 turnos y un resumen cada 6 turnos,
por sesión y por NPC. game/gossip.py: cada pista nueva crea un chisme para
los otros NPC.
6. safety/filters.py para entrada y salida, y caché de respuestas para el
modo demo (DEMO_MODE=1).
7. POST /accuse con los 3 finales. Extra: POST /stt con faster-whisper y GET
/tts con Piper.

Empieza por el punto 1 y espera mi confirmación antes de pasar al siguiente.
```

11. Checklist de arranque (Bloque 0)

Esto se hace en la primera hora y media, antes de separarse.

- [ ] Confirmar con los organizadores que se permiten modelos locales y que el arte puede generarse con IA durante el evento
- [ ] Crear el repositorio `fbclabs/leyendas-sv` e invitar al compañero
- [ ] Instalar Python 3.12, Git y Ollama en la PC con la RTX
- [ ] `ollama pull nemotron-3-nano:4b` y `ollama pull hf.co/bartowski/nvidia_NVIDIA-Nemotron-Nano-9B-v2-GGUF:Q4_K_M`
- [ ] Crear cuenta en build.nvidia.com, generar la clave y guardarla solo en `.env`
- [ ] Correr la prueba de arranque (10 preguntas a la Siguanaba) y elegir el modelo
- [ ] Escribir juntos `content/mystery.yaml` (30 min): verdad, culpable, 6 pistas, 3 finales
- [ ] Escribir juntos `shared/contracts.py` y `docs/api.md` (30 min)
- [ ] Crear el esqueleto de carpetas, `.env.example`, `.gitignore` y `README.md`, y subirlo a `main`

- [ ] Instalar ComfyUI con SDXL y generar un retrato de prueba para fijar el estilo visual
- [ ] Descargar las voces Piper `es_MX-claude-high` y `es_MX-ald-medium` (solo si van por la voz)
- [ ] Abrir los dos chats nuevos con los prompts de la sección 10